Requirements:

1. **File System:** The program shall simulate an ext2 File System. The File System is the structure that shall contain the data.
    1.1. The FS shall use iNodes to represent files.
        1.1.1. The FS shall track which iNodes are available for allocation.
        1.1.2. Each iNode shall contain the block address of the data it represents.
            1.1.2.1. iNodes shall use data blocks (defined in 1.2) to store data.
            1.1.2.2. The iNodes shall use only double indirection and no other methods of addressing.
                1.1.2.2.1. When a file has allocated data blocks, its iNode shall contain the address of a first-level index block.
                1.1.2.2.2. Each used entry in the first-level index block shall contain the address of a second-level index block.
                1.1.2.2.3. Each used entry in the second-level index block shall contain the address of a data block containing the file's contents.
            1.1.2.3. An empty, newly created iNode shall have no data blocks allocated to it.
        1.1.3. Each iNode shall contain the last-modified date of the file contents.
        1.1.4. The iNode shall contain the file type, either file or directory.
        1.1.5. The iNode shall contain the file's protection bits.
        1.1.6. The iNode shall contain the number of directory entries that reference it. This is to enable efficient deletion behavior.
        1.1.7. The iNode shall contain the file owner's user ID.
        1.1.8. The iNode shall contain the file's group ID.
        1.1.9. The iNode shall contain the file's size in bytes.
        1.1.10. The iNode shall contain the file's last access time.
        1.1.11. The iNode shall contain its own last modified date. This is separate from the access or modification dates of the file contents.
        1.1.12. The iNode shall contain the time at which the file was created.
        1.1.13. The iNode shall contain a counter to denote how many times it has been allocated. This is data about the iNode itself and not the file contents.
    1.2. Storage shall be sectioned in units called blocks.
        1.2.1. The FS must track which data blocks are available for allocation.
        1.2.2. Each data block shall have a size of 4KB.
        1.2.3. The FS shall support the allocation of free data blocks.
        1.2.4. The FS shall support the release of an allocated data block for reuse.
        1.2.5. The FS shall support writing data to a block specified by the user.

1.2.6. The FS shall support returning the data in a block specified by the user.

1.2.7. A newly created empty file shall have no allocated data blocks.

1.3. File System Limitations:

1.3.1. The FS shall be sized at 2GB of raw storage.

1.3.2. The FS shall be contained within a singular file on the host machine.

1.3.3. The FS shall maintain the total number of iNodes in the FS.

1.3.4. The FS shall maintain the total number of data blocks in the FS.

1.4. Directories: *The FS shall have the ability to manage and utilize directories.*

1.4.1. Each directory shall be represented by an iNode of the appropriate file type.

1.4.2. Each directory shall support the ability to contain files within.

1.4.3. Directories may be nested within directories to facilitate a tree structure.

1.4.4. Directories shall support the ability to return their own metadata and that of the files within. (Explained in 3.1.1).

1.4.5. Directories shall be made up of directory entries. A directory entry shall contain a file-name paired with its respective iNode.

1.4.6. Directory shall be able to list itself for seeing the entries and files in the list.

1.4.7. Directory shall be accessible for traversing the file system.

1.4.7.1. The FS shall allow the current working directory to be changed using an absolute path beginning at the root directory.

1.4.7.2. The FS shall allow the current working directory to be changed using a relative path starting at the current working directory.

1.4.7.3. The FS shall allow the current working directory to be changed to its parent directory.

1.4.7.4. If the current working directory is the root directory, a request to move to its parent shall leave the current working directory unchanged.

1.4.7.5. If the requested directory does not exist, FS shall return an error with the cause.

1.5. Root Directory Properties:

1.5.1. The root directory shall be known by “/”.

1.5.2. The root directory shall use iNode 0.

1.5.3. The root directory’s initial owner user ID shall be 0.

1.5.4. The root directory’s initial group ID shall be 0.

1.5.5. The root directory’s initial permissions shall be 0777.

1.5.6. The root directory of a newly created FS shall contain no files or subdirectories initially.

1.6. Manage Files: *The FS shall support creating, accessing, modifying and removing files.*

1.6.1. Create files: *The FS shall create a new empty file with the name and location given by the user.*

1.6.1.1. Prohibit duplicates: *The FS shall prohibit two entries from having the same name in the same directory. It can have the same name in a different directory.*

1.6.1.1.1. Locate file: *The FS shall be able to locate a file by name in a given directory.*

1.6.1.1.2. Duplicate error: *If a file with the given file-name already exists, the FS shall be able to exit file creation with appropriate error.*

1.6.1.2. Assign iNode: *The FS shall be able to find the first unused iNode to assign to the file.*

1.6.1.3. Initialize iNode: *The FS shall populate the iNode with the appropriate file info, such as: file name, ID, file type, location of file, file size, protection levels, creation date, and owner UID (as described in section 1.1).*

1.6.1.4. Allocate space: *The FS shall be able to assign unused storage space to the file.*

1.6.1.5. Create directory entry: *The FS shall create a new directory entry with the file name and its assigned iNode (as described in section 1.4.6).*

1.6.2. Delete files: *The FS shall be able remove the file's directory entry and release its storage.*

1.6.2.1. Locate file: *The FS shall be able to locate the file by path (as described in section 1.6.1.1.1).*

1.6.2.2. Release file space: *The FS shall be able to release (deallocate) the space used by the file once the last reference to the file is deleted.*

1.6.2.2.1. Count references: *The FS shall be able to keep track of the number of references to a specific file.*

1.6.2.2.2. Remove directory entry: *The FS shall be able to remove the file's directory entry from the directory.*

2. File System Process:

2.1. Initialization: *Upon request, the program shall create the FS. The FS shall either create a new backing store or read from an existing one. The initial state of the FS shall follow the subsequent constraints:*

2.1.1. The FS Process shall accept arguments on boot that specify its behavior.

2.1.1.1. The FS initialization call shall accept the optional -p flag followed by the port number.

2.1.1.2. If the -p flag is not given, the default port shall be 8080

2.1.1.3. The FS initialization call shall accept the optional -d flag, followed by an integer which denotes the debug level.

2.1.1.4. If the -d flag is not given, the default debug level shall be 0

2.1.2. New backing store: *If the -c flag is provided, the FS shall create a new backing store, according to the following specifications:*

2.1.2.1. The new backing store shall be represented by the name given by the -f flag.

2.1.2.2. Flags: *The program shall require the following flags if creating a new backing store:*

2.1.2.2.1. The -i flag, followed by a decimal number which represents the number of iNodes to use.

2.1.2.2.2. The -b flag, followed by a decimal number which represents the block size in K (1024).

2.1.2.3. The FS shall store data in contiguous data blocks.

2.1.2.4. The FS shall allocate sufficient space to store the appropriate amount of iNodes.

2.1.2.5. File descriptor table: *The FS shall create an empty file descriptor table to store handles to all open files and indexes to those file's iNodes, along with open file information such as the mode each file was opened with.*

2.1.2.5.1. This list must be mutable.

2.1.2.5.2. This list shall keep a counter of the number of open files.

2.1.2.5.3. This list shall contain one entry object per open file.

2.1.2.5.4. This entry shall contain a file handle that points to its iNode.

2.1.2.5.5. This entry shall store the access mode given by the user. Valid modes are read, write, and append.

2.1.2.5.6. Only one of these modes may be active at a time.

2.1.2.5.7. This entry shall store a pointer to designate the current position in the file

2.1.2.5.8. This pointer shall be initialized to point to the start of the file.

2.1.3. Existing backing store: *If the -c flag is not provided, the FS shall find an existing backing store to create the FS from.*

2.1.3.1. Find backing store: *The program shall search for the backing store under the name given by the -f flag.*

2.1.3.1.1. If the specified backing store cannot be found, the FS process shall abort and display an appropriate error message.

2.1.3.2. Load backing store: *The backing store shall be loaded into memory so that the FS recreates the exact state it was in immediately before the last termination of that backing store.*

2.1.3.2.1. File descriptor table: *The file descriptors from the backing store shall be loaded into the file descriptor table of the FS program to keep handles to all in-use files.*

2.1.3.2.2. Working directory: *The working directory path found in the backing store shall be loaded into the FS and used as the current working directory.*

2.1.3.2.3. iNodes: *The FS Process shall load all iNodes from the backing store into memory*

2.1.3.2.4. Data blocks: *The FS Process shall load all data blocks from the backing store into memory.*

2.1.4. The FS Process must support the ability to load all data from a backing store into memory for immediate use. This is done every time the FS process is initialized, when the FS is newly created and when it already exists.

2.2. The FS process shall support opening files for read or write operations

2.2.1. The FS process shall use the open file descriptor table outlined in 2.1.2.5

2.2.2. When an open command is issued by the user, several checks must be performed before fulfilling the request.

2.2.2.1. The FS Process must ensure that all required arguments have been supplied.

2.2.2.1.1. The file name must be provided

2.2.2.1.2. A mode must be provided: read, write, or append.

2.2.2.1.3. The FS Process shall support an optional create flag (-c) that specifies whether the file is to be created

2.2.2.2. Unless the file is to be created, the file must be located in the directory using the provided file name.

2.2.2.2.1. If the file cannot be located, an error shall be thrown to indicate so.

2.2.2.3. If the designated create flag is specified with the open command, then a new file shall be created using the given name.

2.2.2.4. If the file already exists and the create flag is specified, then the process shall abort and return an error indicating the cause. We do not allow duplicate file names within the same directory.

2.2.2.5. The user's permissions shall be checked against the file permissions.

2.2.2.6. If a user's permissions are not sufficient for the mode specified, an error shall be thrown indicating the cause.

2.2.2.7. If the file is empty and the user has requested to read the file, an error shall be thrown indicating the cause.

2.2.2.8. Once a file is successfully opened, the metadata stored within the iNode shall be updated accordingly.

2.2.2.8.1. The access date field shall be updated to the current time.

2.2.2.8.2. If the contents of the file were modified, the last modified field shall be updated to the current date.

2.2.2.8.3. If the contents of the file were modified, the file size shall be updated to reflect the new value.

2.2.3. The FS shall not allow the contents of a read-mode file to be changed.

2.3. Termination: *When told to shut down, the FS program shall terminate cleanly. Upon starting again, the FS shall be in the same state as it was when it shut down.*

2.3.1. The FS Process **shall not** allow the user's data to be corrupted.

2.3.2. Potential crashing conditions must be handled gracefully to avoid premature termination.

2.3.2.1. Although this can only be completely solved through rigorous testing, measures must be taken to protect critical processes from errors.

2.3.3. Save state: *The FS program shall write all data in memory to its backing store.*

2.3.3.1. The FS shall write each file descriptor in the file descriptor table to the reserved location in the backing store.

2.3.3.2. The FS shall write the current working directory path to the reserved location in the backing store.

2.3.3.3. The FS Process shall write all allocated iNodes into the backing store

2.3.3.4. The FS Process shall then write all allocated Blocks into the backing store

2.3.4. Once all data has been safely written to storage, the memory containing the data shall be deallocated.

2.3.5. After all above steps have been completed, the FS Process shall cease operation and close.

2.4. Shell Communication: *The FS Process shall be able to communicate with the shell (the user interface) as described in Section 3.*

2.4.1. The FS process shall support TCP/IP connectivity for wireless connection to a user interface.

2.4.2. The FS process shall support the ability to receive commands

2.4.2.1. The FS process shall support the ability to receive commands through the TCP/IP socket

2.4.2.2. The FS process shall support the ability to receive commands from a local shell process. For development purposes, it is necessary to have a shell

that runs on the same machine as the FS process and uses local communication.

2.4.3. The FS process shall support the ability to execute commands that are given by a shell.

2.4.4. The FS process shall support the ability to return data to a shell program. For example, after a file read operation, the data that was copied from the file must be given to the user somehow.

3. Shell - The User Interface

3.1. The shell shall provide the following functionality to the user:

3.1.1. The ability to list the contents of the current directory.

3.1.1.1. The command that provides this functionality shall imitate the behavior of the "ls -l" command in Unix.

3.1.1.2. The command shall display the name of the file.

3.1.1.3. The command shall display the permission bits of the file.

3.1.1.4. The command shall display the number of references pointing to the file.

3.1.1.5. The command shall display the owner of the file, which is often the same as the creator of a file.

3.1.1.6. The command shall display the permission group of the file.

3.1.1.7. The command shall give the file size in bytes.

3.1.1.8. The command shall give the file's last modified date.

3.1.2. The ability to change the present working directory

3.1.2.1. The command that provides this functionality shall be called "my_cd".

3.1.2.2. cd shall allow the user to advance the present working directory by providing a relative path. A relative path is navigated by appending the given file path to the present working directory path.

3.1.2.2.1. If the relative path provided does not exist, cd shall throw an error indicating the cause.

3.1.2.3. cd shall allow the user to shift the present working directory to the parent directory.

3.1.2.3.1. If the present working directory is the root, or '/', then cd shall do nothing. There must be a parent directory to navigate into.

3.1.2.4. cd shall allow file system traversal via an absolute path. An absolute path is one whose starting directory is the root.

3.1.2.4.1. If the absolute path provided does not exist, cd shall throw an error indicating the cause.

3.1.3. The ability to create a directory: *The FS shall be able to create a subdirectory given a path and a mode. This directory will be represented by a directory entry*

within the FS.

3.1.3.1. Assign iNode: *The directory shall be assigned and iNode which points to the allocated data blocks and holds the mode information of the new directory.*

3.1.3.1.1. The FS shall locate the first available iNode.

3.1.3.1.2. The FS shall allocate the unused iNode to the new directory.

3.1.3.1.3. The iNode shall hold the default mode bits for directories: 0777.

3.1.3.1.4. The iNode shall reference the data block in which the directory will store files as directory entries.

3.1.3.2. New directory entry: *The created directory shall be represented as a directory entry in the directory block specified by the given path. The directory entry's name is the last word in the given path.*

3.1.3.2.1. The FS shall follow the given path to get to the data block which holds the directory entries of the directory specified by all but the last word in the given path.

3.1.3.2.2. The FS shall make a new directory entry in this data block, containing the last word of the path as the file-name and the assigned iNode number.

3.1.4. The ability to remove a directory: *The FS shall be able to remove a subdirectory given a path. Removing a directory will recursively remove every directory and file contained within.*

3.1.4.1. The FS shall follow the given path to the data block which holds the directory entries of the specified directory.

3.1.4.2. Recursive deletion: *The FS shall iterate through each directory entry by removing every file it comes across, and recursively removing each directory it comes across. When a subdirectory has been cleared, it returns back to the parent directory and continues removing each entry. Recursion stops when the initial directory is cleared.*

3.1.4.2.1. The FS shall iteratively access each iNode in the directory entries of the data block.

3.1.4.2.2. The FS shall get the type of entry from the iNode.

3.1.4.2.3. Removing directories: *If the entry is a directory, all of its contents must be deleted. This is accomplished through recursion.*

3.1.4.2.3.1. The FS shall run the remove directory call using the iNode in the entry.

3.1.4.2.4. Removing files: *If the entry is a file, the reference count determines whether the space will be freed or the count decremented (as*

described in section 1.6.2).

3.1.4.2.4.1. The FS shall get the reference count from the iNode

3.1.4.2.4.2. If the reference count is greater than one, decrement the count without freeing data blocks or iNode.

3.1.4.2.5. The FS shall mark as free the data blocks indexed in the iNode.

3.1.4.2.6. The FS shall mark the iNode of the entry as unallocated.

3.1.4.3. The FS shall mark as free the data block indexed by the iNode of the directory at the given path.

3.1.4.4. The FS shall mark this iNode as unallocated.

3.1.4.5. The FS shall remove the directory entry from the parent directory.

3.2. The shell shall include a TCP/IP socket to connect to the file system over the internet.

3.3. The shell shall be responsible for booting the FS process.

3.3.1. Shutdown: *The shell shall support a shutdown command.*

3.3.1.1. The shell shall cleanly terminate the FS process (described in section 2.3)

3.3.1.2. Once the FS process has been terminated, the program shall exit the shell process.

3.3.2. Exit: *The shell shall support an exit command.*

3.3.2.1. The shell shall terminate the FS process (2.3).

3.3.2.2. The shell shall remain open for use with another FS.

Requirement	Section	Summary
Level 1:		
File System	1.	Simulation of ext2 FS. The Data Structure.
File System Process	2.	Operating System that interfaces with the FS.
Shell (User Interface)	3.	Simple user interface that connects to FS process
Level 2:		
iNode File Representation	1.1.	iNode policies, behaviors, and metadata
Block Storage	1.2.	Storage allocation: specifications and alloc policies
FS Limitations	1.3.	Constraints of this FS implementation
Manage Directories	1.4.	FS directory tree policy
Root Directory	1.5.	Properties of the root directory
Manage Files	1.6.	Basic file operations: functionality and policy
FS Initialization	2.1.	Boots FS from new or existing backing store
Opening Files	2.2.	Preparation of files for access or manipulations
Termination	2.3.	Shutdown routine, saving data
Shell Communication	2.4.	Sending and receiving data between shell program
Shell Functionality	3.1.	Full suite of functionality provided by shell
TCP/IP Socket	3.2.	Facilitates internet connection with FS program
Booting FS Process	3.3.	Definitions for FS boot control
Level 3:		
Available iNodes	1.1.1.	The FS knows which iNodes are free or in use
Block Address	1.1.2.	Each iNode must have the address of its data
Last-Modified Date	1.1.3.	Last time the file contents were changed
File Type	1.1.4.	An iNode can represent a file or directory
Protection Bits	1.1.5.	Permissions for file protection
Reference Count	1.1.6.	Count references to self for efficient deletion
Owner’s UID	1.1.7.	User ID of owner
File’s GID	1.1.8.	Group ID of owner
File Size	1.1.9.	Size of file contents in bytes
Access Time	1.1.10.	Keeps track of last time file was opened
iNode Modification Date	1.1.11.	Keeps track of last time the iNode itself was modified
File Creation Time	1.1.12.	Keeps track of the file's creation date
iNode Generation	1.1.13.	Count of times the iNode object itself has been allocated
Available Data Blocks	1.2.1.	FS keeps track of free/used data blocks
4KB Block Size	1.2.3.	Each block unit is 4KB in size

Allocate Data Blocks	1.2.4.	FS reserves storage for files
Release Data Blocks	1.2.5.	FS can free storage for reuse elsewhere
Write to Data Block	1.2.6.	FS is solely responsible for writing data into storage
Return Data	1.2.7.	FS returns data to FS process upon request
Empty File	1.2.8.	Empty files don't begin with allocated storage
ext2 FS Imitation	1.3.1.	The FS shall emulate an ext2 file system
2GB Raw Storage	1.3.2.	The FS will have 2GB of simulated disk space
Single File Container	1.3.3.	The entire FS simulation will be contained in one real file
Number of iNodes	1.3.4.	The FS manages the number of iNodes
Number of Data Blocks	1.3.5.	The FS manages the number of Blocks
iNode Directory Representation	1.4.1.	Directories are also represented by iNodes
Cointain Files	1.4.2.	Directories can hold files and directories within
Tree Structure	1.4.3.	The directory structure conforms to a tree style structure
Return Metadata	1.4.4.	Has access to children's metadata to support commands
Directory Entries	1.4.5.	Specifications for child representation
List Directory	1.4.6.	Can give a summary of itself and children
Traversing Directories	1.4.7.	Directories support shell access for easy navigation
Root Dir Name	1.5.1.	Name of root directory is “/”
Root Dir iNode	1.5.2.	Root directory uses iNode 0. That is, the first in storage
Root Dir Owner UID	1.5.3.	Root directory does not have a user owner, so 0
Root Dir GID	1.5.4.	The root directory's initial group ID shall be 0 as well
Root Dir Permissions	1.5.5.	Specification of default permissions for root
Empty Root Dir	1.5.6.	Root directory is empty when newly created
Create Files	1.6.1.	FS can create files
Delete Files	1.6.2.	FS can delete files
Boot Arguments	2.1.1.	FS Process accepts arguments that specify behavior
New Backing Store	2.1.2.	Specifications for creation of a new, blank FS
Existing Backing Store	2.1.3.	Specifications for how to load existing FS
Load Data from Backing Store	2.1.4.	Loading FS into memory for immediate use
Use File Descriptor Table	2.2.1.	Table for keeping track of open files
Check Before File Open	2.2.2.	Make sure everything is legal when opening a file
Prohibit Changes to read mode	2.2.3.	Files open for read may not have their contents changed
Prevent Data Corruption	2.3.1.	Take measures to protect saved and unsaved data
Handle Crashes	2.3.2.	Crashes must be contained to protect the FS process
Save State	2.3.3.	All data is to be saved to the backing store
Deallocate Memory	2.3.4.	Clear out all cached FS data before closing

Close FS	2.3.5.	Terminate the process
TCP/IP Connectivity	2.4.1.	Support for wireless shell communication
Receive Commands	2.4.2.	Listen to the communication port for instructions
Execute Commands	2.4.3.	Immediately begin execution of commands
Return Data to Shell	2.4.4.	Send data back over the communication line.
List Directory Contents	3.1.1	Summarize each file in a directory
Change Working Directory	3.1.2.	Move present working directory to a new location
Create Directory	3.1.3.	Create a new directory in a specified location
Remove Directory	3.1.4.	Destroy a directory and the denizens within
Shutdown Shell	3.3.1.	Close the shell program
Exit FS Process	3.3.2.	Issue a termination command to the linked FS

Level 4:

Store Data in Blocks	1.1.2.1	iNodes use blocks for data storage
Double Indirection	1.1.2.2.	Specification of Block addressing
No Blocks for New iNode	1.1.2.3.	New iNodes are not assigned Blocks
Change Dir with Absolute Path	1.4.7.1	Begins at root of FS and follows path down to file
Change Dir with Relative Path	1.4.7.2.	Defines ability to navigate across given path
Change to Parent Directory	1.4.7.3.	Defines ability to move upwards in directory tree
Parent of Root Behavior	1.4.7.4.	If attempting to move upward in root, does nothing
Nonexistent Directory Error	1.4.7.5	Directory cannot be navigable unless it exists
Prohibit Duplicates	1.6.1.1.	Two files in the same directory can't have the same name
Assign iNode (File)	1.6.1.2.	Policy for iNode allocation during file creation
Initialize iNode	1.6.1.3.	Mass assignment of metadata upon file creation
Allocate Space	1.6.1.4.	FS can assign unused blocks to files
Create Directory Entry	1.6.1.5.	File creation protocol in relation to directory
Locate File	1.6.2.1.	Functionality to locate files
Release File Space	1.6.2.2.	File deletion protocol
Optional Port #	2.1.1.1.	A communication port may be specified in the shell
Default Port #	2.1.1.2.	The port number to default to
Optional Debug Level	2.1.1.3.	Specification of debug behavior
Default Debug Level	2.1.1.4.	Default debug mode
New Backing Store Name	2.1.2.1.	Default name of backing store file
Flags	2.1.2.2.	Flags required when creating a new backing store

Contiguous Data Blocks	2.1.2.3.	Data blocks belonging to the same file are contiguous
Space for iNodes	2.1.2.4.	The backing store has space reserved for the iNodes
File Descriptor Table	2.1.2.5.	The backing store has space reserved for the FDT
Find Backing Store	2.1.3.1.	Locate backing store by file name
Load Backing Store	2.1.3.2.	Load a backing store into memory
Ensure Arguments Supplied	2.2.2.1.	Throw an error if required arguments are left blank
Existing File Behavior	2.2.2.2.	Existing files must be located before opening
Create Flag Behavior	2.2.2.3.	Open command will create a file if flag is given
Create Duplicate Attempt Error	2.2.2.4.	Throws error if existing file is asked to be created
Permission Check	2.2.2.5.	Sees if you can have access to a resource in the system.
Permission Error	2.2.2.6.	Throws error if permissions are not sufficient
Read Empty File Error	2.2.2.7.	Empty files cannot be opened for read
Open File Update Metadata	2.2.2.8.	Update required fields when a file is opened
Protect from Crashing	2.3.2.1.	Critical processes must be shielded from errors
Save File Descriptor Entries	2.3.3.1.	The exact state of the open file table is to be saved
Save Current Directory	2.3.3.2.	The present working directory is to be saved
Write Allocated iNodes	2.3.3.3.	All iNodes in use shall be saved to the backing store
Write Allocated Blocks	2.3.3.4.	All Blocks in use shall be saved to the backing store
Receive from TCP/IP Socket	2.4.2.1	The process receives commands over the TCP/IP socket
Receive from Shell Process	2.4.2.2	The process can be used via a local shell
UNIX “ls -l” Imitation	3.1.1.1.	The contents of a directory can be summarized
Display File-Name	3.1.1.2.	Gives the name of the file
Display Permissions	3.1.1.3.	Shows what privileges each user has.
Display Reference Count	3.1.1.4.	Shows who has access and what permission they have.
Display File Owner	3.1.1.5.	Gives user ID of file owner
Display Permission Group	3.1.1.6.	Group ID associated with the file
Give File Size	3.1.1.7.	Displays the file size in bytes
Give Last-Modified Date	3.1.1.8.	Gives the file's last-modified field
“my_cd” Command	3.1.2.1.	The name of the command to change working directory
Advance by Relative Path	3.1.2.2.	The working directory may be traversed via a given path
Shift to Parent Directory	3.1.2.3.	The working directory path may be traversed upward
Absolute Path Traversal	3.1.2.4.	A path from root to a navigable directory may be given
Assign iNode (Directory)	3.1.3.1.	Assign iNode to directory pointing to allocated blocks

New Directory Entry	3.1.3.2.	Creates a new path in the file system.
Follow Given Path	3.1.4.1.	Specification for traversing a file path
Recursive Deletion	3.1.4.2.	Procedure for directory removal
Mark Data Block as Free	3.1.4.3.	Deallocate an iNode's data block
Mark iNode as Unallocated	3.1.4.4.	Mark block as unused
Remove Entry from Parent Dir	3.1.4.5.	Removes directory entry within parent directory iNode
Cleanly Terminate FS Process	3.3.1.1.	Ensure all live data is saved
Exit Shell Process	3.3.1.2.	Terminate the shell process
Terminate FS	3.3.2.1.	Terminate the connected FS
Keep Shell Open	3.3.2.2.	Issue a shutdown command while keeping shell open

Level 5:

First-Level Index Block	1.1.2.2.1.	A file’s iNode references a first level index block
Second-Level Index Block	1.1.2.2.2.	A first level index block references a second level block
File Content Data Block	1.1.2.2.3.	A second level index block contains index of the data
Locate File	1.6.1.1.1.	Locate a file given a path name (directory + file name)
Duplicate Error	1.6.1.1.2.	If a file with given name is found in directory, throw error
Count References	1.6.2.2.1.	Keep count of number of references to each file
Remove Directory Entry	1.6.2.2.2.	Remove the file’s directory entry from the directory
-i Flag	2.1.2.2.1.	Number after -i flag represents number of iNodes to use
-b Flag	2.1.2.2.2.	Number after -b flag represents block size in K (1024)
Mutability of FDT	2.1.2.5.1.	File Descriptor Table (FDT) must be mutable
Open File Count	2.1.2.5.2.	FDT keeps count of open files
Per Open-File Entry Object	2.1.2.5.3.	FDT contains one entry object per open file
File Handle to iNode	2.1.2.5.4.	Entry contains file handle pointing to the file’s iNode
Access Mode	2.1.2.5.5.	Entry stores access mode (read/write/append)
Exclusivity of Modes	2.1.2.5.6.	Only one mode may be active at a time
File Position Pointer (FPP)	2.1.2.5.7.	Entry stores “pointer” to designate current position in file
Initial FPP Behavior	2.1.2.5.8.	FPP initially points to the start of the file
Error: Backing Store Not Found	2.1.3.1.1.	Aborts and displays error if backing store doesn’t exist
File Descriptor Table (Load)	2.1.3.2.1.	Load the file descriptors from the backing store into FDT
Working Directory (Load)	2.1.3.2.2.	Load the working directory from the backing store into FS

Load iNodes	2.1.3.2.3.	Load all iNodes from backing store into memory
Load Data Blocks	2.1.3.2.4.	Load all data blocks from the backing store into memory
Provide File Name	2.2.2.1.1.	The file name must be provided
Provide Mode	2.2.2.1.2.	A mode must be provided (read/write/append)
Optional Create Flag	2.2.2.1.3.	Supports optional create flag (should file be created?)
File Location Error	2.2.2.2.2.	Displays error if file cannot be located
Update Access Date Field	2.2.2.8.1.	The access date field shall be updated to the current time
Update Last-Modified Field	2.2.2.8.2.	Set last-modified field to current date if file was modified
Update File Size	2.2.2.8.3.	Update file size if file content was modified
Error: Relative Path DNE	3.1.2.2.1.	Throws error if given relative path does not exist
“my_cd” on Parent Directory	3.1.2.3.1.	Do nothing if attempting to navigate to parent of root
Error: Absolute Path DNE	3.1.2.4.1.	Displays error if given absolute path does not exist
Locate First Available iNode	3.1.3.1.1.	The FS shall locate the first available iNode
Allocate iNode (Directory)	3.1.3.1.2.	The FS shall allocate the unused iNode to the directory
Default Mode (Directory)	3.1.3.1.3.	The iNode holds the default mode for directories (0777)
iNode Reference to Data Block	3.1.3.1.4.	iNode references block where directory entries are kept
Follow Path (New Directory)	3.1.3.2.1.	Follow path to block holding desired directory entries
Directory Entry (New Directory)	3.1.3.2.2.	Make a new directory entry in data block with file-name
Iterative iNode Access	3.1.4.2.1.	Iteratively access each iNode of files in directory
Get Entry Type	3.1.4.2.2.	The FS shall get the type of entry from the iNode
Removing Directories	3.1.4.2.3.	Remove directory entry and its contents recursively
Removing Files	3.1.4.2.4.	Check if file needs removing based on reference count
Mark Data Blocks as Free	3.1.4.2.5.	Mark as free the data blocks indexed by the iNode
Mark iNode as Unallocated	3.1.4.2.6.	Mark the iNode of the entry as unallocated

Level 6:

Recursive Removal	3.1.4.2.3.1.	Run remove directory using iNode of entry
Get Reference Count	3.1.4.2.4.1.	The FS shall get the reference count from the iNode
Decrement Count	3.1.4.2.4.2.	Decrement counts > 1 without freeing iNode or Block